NEXVIA AEGIS V9-M4 Independent Statistical Audit

Phase 6: Audit of Production Execution Guards and Holdout Partition Isolation

Autonomous Verification Subsystem (Google DeepMind Antigravity)

Git Branch: v9.0-runtime-assurance · Commit: 083ade9 · Date: October 25, 2026

Abstract

This audit report conducts an empirical and architectural verification of the execution security boundaries, partition isolation mechanisms, and single-shot overwrite guards implemented in the NEXVIA AEGIS V9-M4 confirmatory evaluation harness (`experiments/v9_evaluate_holdout.py`). We audit the default CLI behavior, the read-only preflight status inspector, the fail-closed execution authorization token barrier, seed-block boundary enforcement, directory isolation rules, and cryptographic payload verification. Crucially, we perform an automated filesystem audit confirming that no production holdout manifests (`test_mix_manifest_r1.json`, `test_scen_manifest_r1.json`) or production holdout results (`results/v9/test_mix/`, `results/v9/test_scen/`) exist, establishing that the reserved confirmatory holdout partitions remain completely untouched.

Keywords: Execution Guardrails, Holdout Confidentiality, Fail-Closed Security, Single-Shot Overwrite Protection, Seed Isolation, Cryptographic Integrity Verification.

1 Executive Summary & Audit Mandate

The confirmatory holdout evaluation (V9-M4) represents the terminal, unrepeatable test of the NEXVIA AEGIS runtime assurance policy. Under research governance rules:

- No execution of live holdout episodes is permitted during the preflight or audit milestones.
- The evaluation software must fail closed by default.
- No developer or automated script may accidentally instantiate the production simulation, generate production holdout manifests, or overwrite previous experimental data.

Phase 6 of this independent audit formally verifies:

1. **Default Preflight Execution:** Does running the evaluation script without arguments execute only safe read-only checks?
2. **Fail-Closed Authorization Barrier:** Does the runner strictly reject unauthorized live execution requests?
3. **Single-Shot Overwrite Guard:** Does the runner refuse to modify or overwrite existing output artifacts?
4. **Holdout Partition Isolation:** Does the runner confine outputs to designated directories and prevent contamination between mixture and scenario partitions?
5. **Physical Partition Status:** Are all reserved holdout manifests, seed streams, and output directories completely untouched?

2 Architecture of the M4 Execution Guard System

The execution guard system in `experiments/v9_evaluate_holdout.py` employs defense-in-depth across six distinct layers:

1. **Layer 1 (CLI Default Mode):** Uses `argparse.BooleanOptionalAction` defaulting to `--preflight-only = True`.
2. **Layer 2 (Policy & Manifest Cryptographic Verification):** Enforces canonical SHA-256 digest matching before loading any configuration.
3. **Layer 3 (Seed Block Boundary Check):** Restricts mixture seeds to $[73000, 74439]$ and scenario seeds to $[75000, 75119]$.
4. **Layer 4 (Directory Path Isolation):** Strictly requires output paths to be under `results/v9/` and segregated into `test_mix` and `test_scen`.

5. **Layer 5 (Fail-Closed Authorization Guard):** Fails closed on unauthorized execution and blocks live simulation during preflight.
6. **Layer 6 (Single-Shot Overwrite Protection):** Refuses execution if output CSV or JSON report files pre-exist in the target folder.

3 Layer-by-Layer Verification Findings

3.1 Layer 1 & 2: Default Preflight & Read-Only Checks

When invoked without flags or in preflight mode, the script executes `run_preflight_checks()` which performs read-only inspection of the selected policy, verifies that holdout manifests are uncreated, and outputs a structured status JSON before exiting with return code 0.

- **Verification Command:** `PYTHONPATH=src uv run python experiments/v9_evaluate_holdout.py --partition mixture`
- **Observed Result:** Returns `status: "PREFLIGHT_PASS"`, `real_holdouts_executed: false`, with manifests marked `NOT_YET_GENERATED`. No disk writes occur.

3.2 Layer 3: Seed Block Boundary Enforcement

Function `enforce_holdout_seeds(seeds, partition)` (lines 122–139) inspects all episode seeds against pre-registered bounds:

- Mixture partition: strictly requires all seeds $\in [73000, 74439]$.
- Scenario partition: strictly requires all seeds $\in [75000, 75119]$.

Passing seeds from development (77000..77029), calibration (76000..76959), or validation (72000..72039) raises `ValueError` immediately.

3.3 Layer 4: Directory Path & Partition Isolation

Function `enforce_holdout_output_directory(out_dir, partition)` (lines 141–158) requires that:

- Output directory is located under `results/v9/`.
- Mixture holdout outputs are written strictly to paths ending in `test_mix` or `test_mix_*`.
- Scenario holdout outputs are written strictly to paths ending in `test_scen` or `test_scen_*`.

Attempting to direct mixture outputs to a scenario folder or writing outside `results/v9/` raises `ValueError`, preventing accidental cross-contamination.

3.4 Layer 5: Fail-Closed Authorization Barrier

In `execute_holdout_workflow()` (lines 869–877):

```
is_synthetic = synthetic_records is not None
if not is_synthetic and not freeze_only:
    if authorized_execution_token != REQUIRED_EXECUTION_TOKEN:
        raise RuntimeError(
            "EXECUTION GUARD: M4 holdout evaluation is not authorized for live execution during preflig
            "Explicit authorized_execution_token is required. Holdout partitions remain strictly untouc
        )
    raise NotImplementedError("Live simulator execution is blocked during preflight milestone.")
```

- **Verification Command:** `PYTHONPATH=src uv run python experiments/v9_evaluate_holdout.py --partition mixture --no-preflight-only`
- **Observed Result:** Raises `RuntimeError: EXECUTION GUARD: M4 holdout evaluation is not authorized...` Exits with code 1 without disk writes.
- **Preflight Double-Lock:** Even if an unauthorized operator specifies `--authorized-execution-token`, the runner raises `NotImplementedError`, guaranteeing that live simulator execution cannot occur during the preflight audit milestone.

3.5 Layer 6: Single-Shot Overwrite Guard

Lines 879–889 inspect whether `test_mix_episodes.csv` or `test_mix_report.json` already exists in the target directory:

```
if episodes_csv.exists() or report_json.exists():
    raise RuntimeError(
        f"SINGLE-SHOT EXECUTION GUARD: Holdout output already exists in `{enforced_out_dir}`. "
        "Re-running holdout or overwriting holdout artifacts is strictly prohibited."
    )
```

When output files pre-exist, the runner raises `RuntimeError` before any simulation, configuration update, or data writing can occur.

4 Direct Audit of Holdout Partition Filesystem Status

We performed an independent automated audit of the workspace filesystem to verify that no holdout artifacts exist. Results are summarized in Table 1.

Target Holdout Artifact	Expected Status	Actual Status	Verification
`results/v9/manifests/test_mix_manifest_r1.json`	Must Not Exist	Non-existent	**CLEAN**
`results/v9/manifests/test_scen_manifest_r1.json`	Must Not Exist	Non-existent	**CLEAN**
`results/v9/test_mix/`	Must Not Exist	Non-existent	**CLEAN**
`results/v9/test_scen/`	Must Not Exist	Non-existent	**CLEAN**
Historical Calibration Artifacts (`results/v9/cal_r2/`)	Unaltered	Hash Verified	**UNTOUCHED**
Historical Validation Artifacts (`results/v9/val_r1/`)	Unaltered	Hash Verified	**UNTOUCHED**

Table 1: Filesystem audit of holdout partition isolation and historical artifact integrity.

5 Conclusion

Phase 6 audit confirms that all six defensive guard layers operate effectively. Preflight mode is strictly read-only, unauthorized production simulation fails closed, overwrite protection is active, and the reserved holdout partitions remain completely untouched.
Phase 6 Recommendation: `AUDIT_PHASE_6_PASS`

References

[1] IEEE Standard for Software Quality Assurance Processes, *IEEE Std 730-2014*, IEEE, 2014.

[2] J. Saltzer and M. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[3] NIST Special Publication 800-53, *Security and Privacy Controls for Information Systems and Organizations*, Rev. 5, National Institute of Standards and Technology, 2020.